

File Guard Final Report

File Guard Final Report

System architecture

File Guard is a modular PyQt6 desktop application with a local Python backend. The GUI handles registration, PIN login, OTP-based PIN reset, file and folder selection, monitoring status, alerts, log viewing, and report generation. The backend is split into small modules for authentication, hashing, storage, monitoring, risk classification, and reporting.

How hash comparison works

The user selects one hashing algorithm: MD5, SHA-1, or SHA-256. When monitoring starts, File Guard recursively scans every selected folder and directly scans every selected file. It calculates a baseline hash for each file and stores that snapshot in a local JSON file. During each monitoring cycle, it recalculates the current hash and compares it with the baseline. If the hash changes, the file is marked as modified. If a baseline file no longer exists, it is marked as deleted. If a new file appears under a monitored folder, it is marked as added.

Risk level logic

Risk levels are based on file extension and path hints.

- Low Risk: text and document-oriented files such as .txt, .md, .csv, .json, and .log.
- Medium Risk: configuration-oriented files such as .ini, .conf, .yaml, .toml, and .env.
- High Risk: executables, system files, key material, databases, or files in sensitive paths such as

Authentication and OTP

On first launch, the user registers a mobile number and a numeric PIN. The mobile number is only used for OTP-based PIN reset. The OTP is simulated for testing by printing it to the console and also showing it in the GUI test message. The OTP entry widget uses one-character fields and automatically shifts focus to the next field after each digit.

Logging and alerts

Every detected event is written to both a JSONL log and a plain text log. Each record includes timestamp, file path, change type, old hash, new hash, and risk level. Real-time alerts are shown in the GUI alerts panel and also pushed to the system tray notification API when available.

Final monitoring reports

File Guard generates two runtime monitoring reports:

- A text report summarizing totals, risk counts, affected files, and timeline.
- A PDF report with the same summary in a printable format.

Cross-platform support

The application uses pathlib, hashlib, and PyQt6 so the same codebase works across Windows and Linux. Path normalization is handled with `pathlib.resolve()`, and recursive folder scanning uses `os.walk()`.

Testing status

Implementation and syntax validation were completed in this development environment. A full manual GUI test should be performed on:

- Windows 10 or Windows 11
- Kali Linux VM

Recommended validation steps:

- Register a new user with mobile number and PIN.
- Log in with the PIN.
- Request an OTP and reset the PIN.
- Select both a file and a folder.
- Start monitoring with each hash algorithm.
- Modify a text file and confirm a Low Risk alert.
- Modify a configuration file and confirm a Medium Risk alert.
- Modify or delete an executable or sensitive file and confirm a High Risk alert.
- Generate text and PDF reports.

Screenshots

Capture the following screenshots during manual execution:

- Login screen
- File selection screen
- Monitoring in progress
- Alert popup or tray notification
- Log viewer
- Final report output